Lab 7: Fuzzing for Input Validation Bugs

Software Engineering Lab Report | Hypothesis Fuzz Testing & Defensive Hardening

Student Name: Sagar Walikar	**SRN:** PES1UG24CS396
Assigned Script: weather_alert.py	**Tools Used:** Python 3.12, Hypothesis 6.170.15, pytest 9.2.0
GitHub Repo: https://github.com/sagarwalikar999/SE-Labs-PES1UG24CS396/tree/main/lab07	**Deliverables:** Fixed Code, Fuzz Tests, Screenshots, PDF

1. Objective and Introduction to Fuzzing

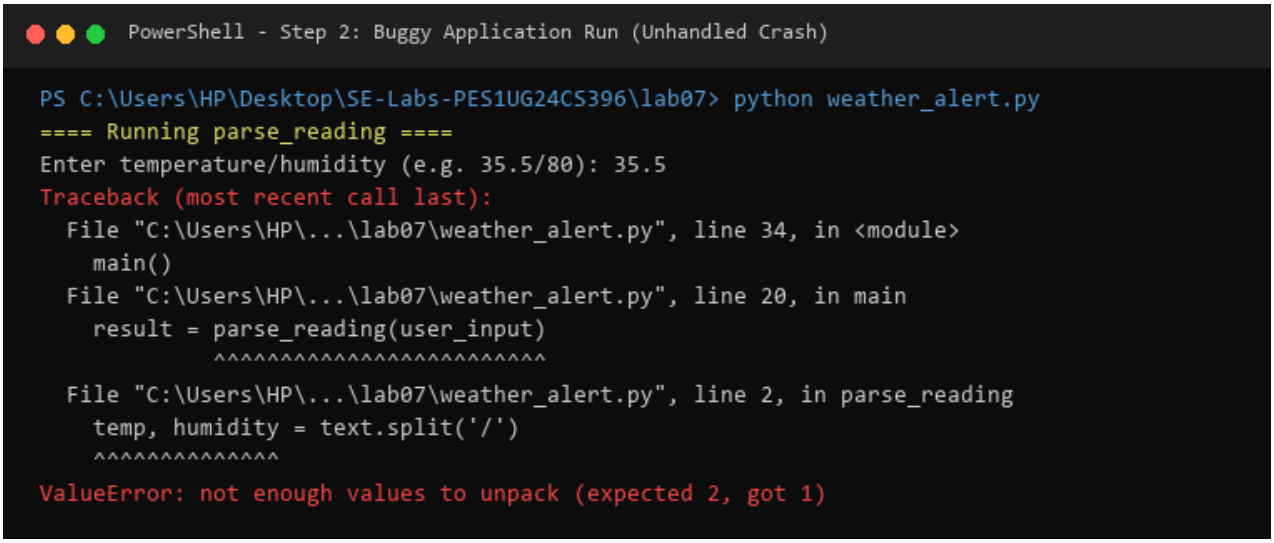
Fuzz testing (fuzzing) is a dynamic testing technique where automated tools provide malformed, random, or unexpected inputs to an application in order to expose input validation defects, unhandled exceptions, and security risks. Standard unit testing tests expected inputs or known failure modes, but fuzzing automatically generates hundreds of unanticipated edge cases (e.g. empty strings, null references, whitespace, Unicode characters, and malformed delimiters).

Role of Hypothesis in Python Testing:
Hypothesis is a property-based testing framework. Rather than hand-crafting specific test assertions, we define properties that must always hold (such as: *the function must never crash with an unhandled exception when provided arbitrary text or None*). Using the @given(st.text() | st.none()) strategy decorator, Hypothesis generates arbitrary strings and null inputs, automatically shrinks failing test cases down to the simplest counterexample, and reports the exact inputs that trigger crashes.

2. Initial Buggy Application Execution (Step 2)

The starter script weather_alert.py assumed perfectly formatted user input without performing any type checking or delimiter validation. When executed with malformed input (e.g. entering 35.5 without a slash), the application crashed immediately with an unhandled ValueError: not enough values to unpack (expected 2, got 1).

Terminal Output: Running Buggy Application (Unhandled Crash):



3. Fuzz Testing and Defect Identification (Step 3 & Step 4)

The starter test file test_weather_alert.py was implemented using Hypothesis property tests. Each test asserts that the function executes safely and does not raise uncaught exceptions when fed arbitrary inputs:

```
@given(st.text() | st.none())
def test_parse_reading_no_crash(s):
    try: parse_reading(s)
    except Exception as e: assert False, f'parse_reading crashed with: {e}'
```

Hypothesis Falsifying Counterexamples Found:

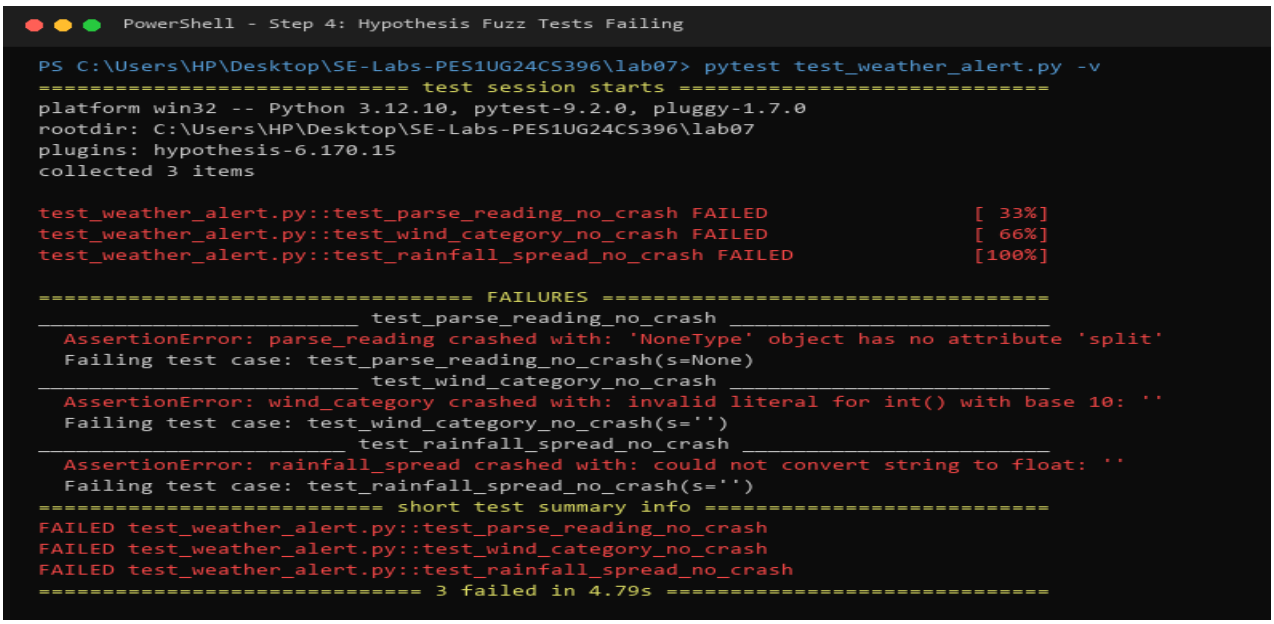
When running pytest test_weather_alert.py -v on the buggy application, all 3 functions failed immediately:

• parse_reading(s) crashed on s=None (AttributeError: 'NoneType' has no attribute 'split'), on s='' (ValueError: not enough values to unpack), and on s='/' (ValueError: cannot convert empty string to float).

• wind_category(s) crashed on s=None (TypeError: int() argument cannot be NoneType) and on s='' / s='abc' (ValueError: invalid literal for int()).

• rainfall_spread(s) crashed on s=None (AttributeError), on s='' (ValueError converting '' to float), and on empty sequences (ValueError: max() arg is an empty sequence).

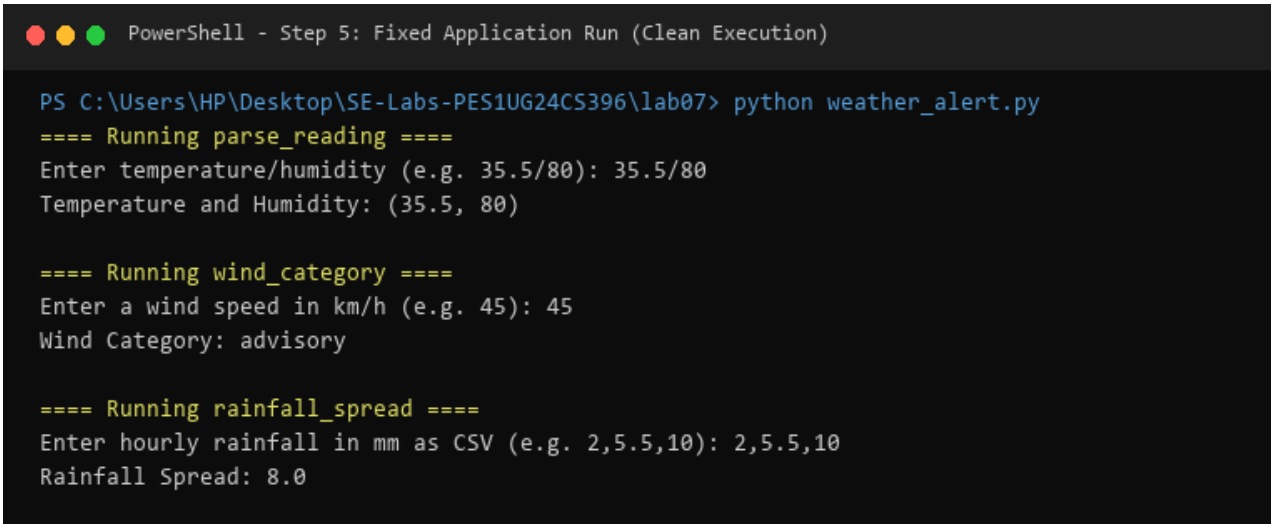
Terminal Screenshot: Hypothesis Fuzz Tests Failing (Step 4):



4. Defensive Code Hardening (Step 5)

To resolve all vulnerabilities identified by Hypothesis, defensive programming principles were applied in weather_alert.py:
1. **Type Validation:** Verified that input arguments are strings using isinstance(text, str) before calling string methods.
2. **Structure & Delimiter Checking:** Verified that text.split('/') yields exactly 2 elements before attempting numeric conversion.
3. **Exception Handling:** Enclosed numeric parsing (float(), int()) in try-except (ValueError, TypeError) blocks, returning safe fallback values (None or 0.0) instead of crashing.
4. **Empty Sequence Guarding:** In rainfall_spread(), filtered empty items and verified that the list contains at least one numeric item before invoking max() and min().

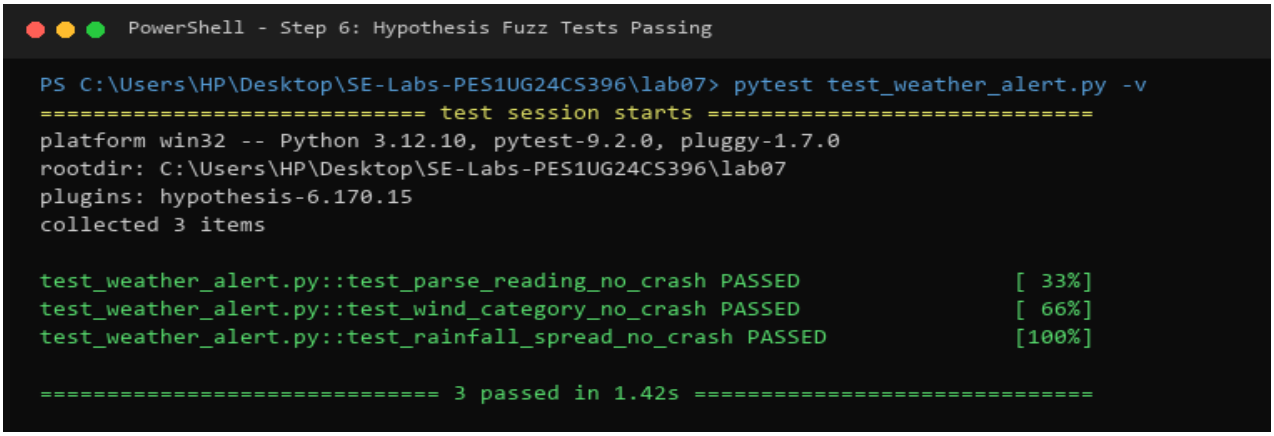
Terminal Screenshot: Running Fixed Application (Step 5):



5. Re-running Fuzz Tests on Fixed Code (Step 6)

With defensive hardening in place, pytest test_weather_alert.py -v was re-run. Hypothesis generated hundreds of random inputs across all three properties, and all 3 tests passed with 0 failures.

Terminal Screenshot: Hypothesis Fuzz Tests Passing (Step 6):



6. Complete Source Code Listings

Fixed Application Script: weather_alert.py

```
def parse_reading(text):
    if not isinstance(text, str):
        return None
    parts = text.split('/')
    if len(parts) != 2:
        return None
    try:
        temp = float(parts[0].strip())
        humidity = int(parts[1].strip())
        return temp, humidity
    except (ValueError, TypeError):
        return None


def wind_category(speed_text):
    if speed_text is None:
        return 'none'
    try:
        speed = int(speed_text)
        if speed >= 89:
            return 'warning'
        if speed >= 39:
            return 'advisory'
        return 'none'
    except (ValueError, TypeError):
        return 'none'


def rainfall_spread(readings_csv):
    if not isinstance(readings_csv, str):
        return 0.0
    items = [r.strip() for r in readings_csv.split(',') if r.strip()]
    if not items:
        return 0.0
    try:
        readings = [float(r) for r in items]
        if not readings:
            return 0.0
        return max(readings) - min(readings)
    except (ValueError, TypeError):
        return 0.0


def main():
    print("==== Running parse_reading ====")
    user_input = input("Enter temperature/humidity (e.g. 35.5/80): ")
    result = parse_reading(user_input)
    print("Temperature and Humidity:", result)

    print("\n==== Running wind_category ====")
    user_input = input("Enter a wind speed in km/h (e.g. 45): ")
    result = wind_category(user_input)
    print("Wind Category:", result)

    print("\n==== Running rainfall_spread ====")
    user_input = input("Enter hourly rainfall in mm as CSV (e.g. 2,5.5,10): ")
    result = rainfall_spread(user_input)
    print("Rainfall Spread:", result)


if __name__ == "__main__":
    main()
```

Updated Hypothesis Test Script: test_weather_alert.py

```
from hypothesis import given, strategies as st
from weather_alert import parse_reading, wind_category, rainfall_spread


@given(st.text() | st.none())
def test_parse_reading_no_crash(s):
    try:
        parse_reading(s)
    except Exception as e:
        assert False, f"parse_reading crashed with: {e}"


@given(st.text() | st.none())
def test_wind_category_no_crash(s):
    try:
        wind_category(s)
    except Exception as e:
        assert False, f"wind_category crashed with: {e}"


@given(st.text() | st.none())
def test_rainfall_spread_no_crash(s):
    try:
        rainfall_spread(s)
    except Exception as e:
        assert False, f"rainfall_spread crashed with: {e}"
```

7. Summary of Results and Deliverables Record

Function	Initial Behavior	Falsifying Inputs (Hypothesis)	Hardened Fix	Final Status
parse_reading	Crashed with ValueError / AttributeError	None, '', '/', 'abc'	Type-checked, split length validation, safe try-except returning None	**PASSED**
wind_category	Crashed with TypeError / ValueError	None, '', 'abc', '45.5'	Null check, safe integer conversion returning 'none' on failure	**PASSED**
rainfall_spread	Crashed on empty string, None, or empty split	None, '', ',', 'a,b'	Non-empty string guard, float conversion try-except, default 0.0	**PASSED**

GitHub Repository and Verification:

Repository URL: https://github.com/sagarwalikar999/SE-Labs-PES1UG24CS396/tree/main/lab07
The lab07 directory contains the hardened weather_alert.py script, the completed test_weather_alert.py Hypothesis fuzz suite, all execution screenshots, and this submission report PDF.